

17

الوحدات: استيرادها وإنشائها

توفر لغة بايثون مجموعة متنوعة من الدوال المضمّنة مثل:

- `print()`: تطبع التعبيرات المُمرّرة إليها في مجرى الخرج
- `abs()`: تُعيد القيمة المطلقة للعدد
- `int()`: تحوّل القيمة المُمرّرة إليها إلى عدد صحيح
- `len()`: تُعيد طول تسلسل أو مجموعة

هذه الدوال المضمّنة مفيدة، لكنها محدودة، لهذا يستخدم المطورون الوحدات (modules)

لتطوير برامج أكثر تعقيدًا. الوحدات (Modules) هي ملفات بايثون ذات امتداد `.py`، والتي تحوي شيفرات بايثون. يمكن التعامل مع أيّ ملف بايثون على أنه وحدة. مثلاً، إن كان هناك ملف بايثون يسمى `hello.py`، فسيكون اسم الوحدة المقابلة له `hello`، والذي يمكن استيراده في ملفات بايثون الأخرى، أو استخدامه في مترجم (interpreter) سطر أوامر بايثون.

يمكن للوحدات أن تعرّف دوالاً وأصنافاً و**متغيرات** يمكن الرجوع إليها من ملفات بايثون

الأخرى، أو من مترجم سطر أوامر بايثون.

في بايثون، يمكنك الوصول إلى الوحدات باستخدام التعليمة `import`. عند فعل ذلك،

سنُنقذ شيفرة الوحدة، مع الاحتفاظ بنطاقات (scopes) التعريفات حتى تكون متاحة في ملفك الحالي. فعندما تستورد بايثون وحدةً باسم `hello` على سبيل المثال، فسيبحث المترجم أولاً عن وحدة مضمّنة باسم `hello`. فإن لم يجد، فسيبحث عن ملف يسمى `hello.py` في قائمة من المجلدات يحددها المتغير `sys.path`.

سيرشدك هذا الفصل إلى كيفية البحث عن الوحدات وتثبيتها، واستيرادها، وإعادة تسميتها

(aliasing)، ثم سيعلمك كيفية إنشاء وحدة كاملة واستعمالها في شيفرة أخرى.

1. تثبيت الوحدات

هناك عدد من الوحدات المضمّنة في مكتبة بايثون القياسية التي تحتوي على العديد من الوحدات التي توفر الكثير من وظائف النظام، أو توفر حلولاً قياسية. مكتبة بايثون القياسية تأتي مع كل توزيعات بايثون.

للتحقق من أنّ وحدات بايثون جاهزة للعمل، ادخل إلى بيئة برمجة بايثون 3 المحلية، أو بيئة البرمجة المستندة إلى الخادم، وشغّل مترجم بايثون في سطر الأوامر على النحو التالي:

```
(my_env) sammy@ubuntu:~/environment$ python
```

من داخل المترجم، يمكنك تنفيذ التعليمة `import` مع اسم الوحدة للتأكد من أنّها جاهزة:

```
import math
```

لأنّ كانت `math` وحدة مضمّنة، فينبغي أن يُكمل المترجم المهمة دون أي مشاكل، ثم يعود إلى المحث (prompt). هذا يعني أنّك لست بحاجة إلى فعل أيّ شيء للبدء في استخدام الوحدة `math`.

لننّفذ الآن التعليمة `import` مع وحدة قد لا تكون مُثَبَّتة عندك، مثل `matplotlib`، وهي مكتبة للرسم ثنائي الأبعاد:

```
import matplotlib
```

إذا لم تكن `matplotlib` مُثَبَّتة، فسيظهر خطأً مثل الخطأ التالي:

```
ImportError: No module named 'matplotlib'
```

يمكنك إيقاف مترجم بايثون بالضغط على `CTRL + D`، ثم تثبيت الوحدة `matplotlib` عبر

pip بتنفيذ الأمر التالي:

```
(my_env) sammy@ubuntu:~/environment$ pip install matplotlib
```

بمجرد تثبيتها، يمكنك استيراد matplotlib من مترجم بايثون باستخدام

import matplotlib، ولن يحدث أي خطأ.

2. استيراد الوحدات

للاستفادة من الدوال الموجودة في الوحدة، ستحتاج إلى استيراد الوحدة عبر التعليلة

import، إذ تتألف من الكلمة المفتاحية import معقوبة باسم الوحدة كما في المثال الآتي.

يُصرّح عن عملية استيراد الوحدات في أعلى ملفات بايثون، قبل الأسطر التوجيهية

shebang lines أي الأسطر التي تبدأ ب #)، أو التعليقات العامة. لذلك، سنستورد في ملف

برنامج بايثون my_rand_int.py الوحدة random لتوليد أعداد عشوائية على النحو التالي:

```
import random
```

عندما نستورد وحدة، نجعلها بذلك متاحة في برنامجنا الحالي بوصفها فضاء اسم

(namespace) منفصل. أي سيتعيّن علينا الوصول إلى الدالة باستخدام الصياغة النقطية

(dot notation) على النحو التالي [function].[module].

عملياً، ستبدو الشيفرة باستخدام مثال الوحدة random كما يلي:

- random.randint(): تستدعي الدالة لإعادة عدد صحيح عشوائي، أو

- random.randrange(): تستدعي الدالة لإعادة عنصر عشوائي من نطاق محدد.

دعنا ننشئ حلقة for لتوضيح كيفية استدعاء دالة من الوحدة random ضمن

البرنامج my_rand_int.py:

```
import random

for i in range(10):
    print(random.randint(1, 25))
```

يستورد هذا البرنامج الصغير الوحدة random في السطر الأول، ثم ينتقل إلى الحلقة for التي ستمر على 10 عناصر. داخل الحلقة، سيطبع البرنامج عددًا صحيحًا عشوائيًا من المجال 1 إلى 25 (مشمول). يُمَرَّر العددين الصحيحان 1 و 25 بوصفهما معاملات إلى random.randint().

عند تنفيذ البرنامج باستخدام الأمر python my_rand_int.py، ستظهر 10 أعداد صحيحة عشوائية في المخرجات. نظرًا لأنَّ هذه العناصر عشوائية، فستحصل على الأرجح على أعداد مختلفة (كلها محصورة بين 1 و 25) في كل مرة تنفِّذ فيها البرنامج، لكنَّها عمومًا ستبدو كما يلي:

```
6
9
1
14
3
22
10
1
15
9
```

إذا رغبت في استخدام دوال من أكثر من وحدة، يمكنك ذلك عن طريق إضافة عدة

تعليقات استيراد:

```
import random
import math
```

قد تصادف شيفرات تستورد عدة وحدات مفصولة بفواصل مثل `import random, math`

ولكن هذا لا يتوافق مع دليل التنسيق [PEP 8](#).

للاستفادة من الوحدة الإضافية، يمكننا إضافة الثابت `pi` من الوحدة `math` إلى برنامجنا،

وتقليل عدد الأعداد الصحيحة العشوائية المطبوعة:

```
import random
import math

for i in range(5):
    print(random.randint(1, 25))

print(math.pi)
```

الآن، عند تنفيذ البرنامج، سنحصل على مخرجات على الشكل التالي، مع تقريب للعدد `pi`

في السطر الأخير:

```
18
10
7
13
10
3.141592653589793
```

تتيح لك التعليمة `import` استيراد وحدة واحدة أو أكثر إلى برامجك، وهذا يمكّنك من الاستفادة مما تحويها تلك الوحدات.

3. استيراد عناصر محدّدة

للإشارة إلى عناصر من وحدة مستوردة ضمن فضاء الأسماء، يمكنك استخدام التعليمة `from ... import`. عندما تستورد الوحدات بهذه الطريقة، سيكون بمقدورك الرجوع إلى الدوال بأسمائها فقط، بدلاً من استخدام الصياغة النقطية. في هذه الصياغة، يمكنك تحديد التعريفات التي تود الإشارة إليها مباشرة.

في بعض البرامج، قد ترى العبارة `from ... import *`، إذ تشير العلامة `*` إلى جميع العناصر الموجودة في الوحدة، ولكن هذه الصياغة غير معتمدة في [PEP 8](#).

سنحاول في البداية استيراد دالة واحدة من الوحدة `random`، وهي `randint()`:

```
from random import randint
```

هنا، نستدعي أولاً الكلمة المفتاحية `from`، ثم `random`. بعد ذلك، نستخدم الكلمة المفتاحية

`import`، ونستدعي الدالة المحددة التي نوّد استخدامها.

الآن، عندما نرغب في استخدام هذه الدالة في برنامجنا، لن نستدعي الدالة وفق الصياغة

النقطية، `random.randint()`، ولكن سنستدعيها باسمها مباشرةً، أي `randint()`:

```
from random import randint
```

```
for i in range(10):
    print(randint(1, 25))
```

عند تنفيذ البرنامج، ستتلقى مخرجات مشابهة لما تلقيته مسبقًا.

يتيح لنا استخدام `import ... from` الرجوع إلى العناصر المعرّفة في الوحدة من فضاء

الأسماء الخاص ببرنامجنا، مما يتيح لنا تجنب استخدام الصياغة النقطية الطويلة.

4. الأسماء المستعارة في الوحدات

يمكن إعطاء أسماء مستعارة للوحدات ودوالها داخل بايثون باستخدام الكلمة

المفتاحية `as`.

قد ترغب في تغيير اسم ما لأُكّ تستخدمه سلفًا في برنامجك، أو أنّه مستخدم في وحدة

أخرى مستوردة، أو قد ترغب في اختصار اسم طويل تستخدمه كثيرًا. يمكنك ذلك عبر

الصياغة التالية:

```
import [module] as [another_name]
```

لنعدّل اسم الوحدة `math` في ملف البرنامج `my_math.py`. سنغيّر اسم الوحدة `math` إلى `m`

من أجل اختصاره. سيبدو برنامجنا المعدل كالتالي:

```
import math as m
```

```
print(m.pi)
```

```
print(m.e)
```

سنشير داخل البرنامج إلى الثابت `pi` بالتعبير `m.pi`، بدلًا من `math.pi`.

يشيع في بعض الوحدات استخدام أسماء مستعارة (`aliases`) محدّدة. فمثلاً، يدعو التوثيق

الرسمي للوحدة `matplotlib.pyplot` إلى استخدام الاسم المستعار `plt`:


```
import matplotlib.pyplot as plt
```

يسمح هذا للمبرمجين بإلحاق الكلمة القصيرة `plt` بأي دالة متاحة داخل الوحدة، كما هو

الحال في `plt.show()`.

5. كتابة وحدات مخصصة واستيرادها

سنتعلم الآن كتابة وحدات بايثون - بعد تعلم كيفية استيرادها - لاستخدامها في ملفات

البرمجة الأخرى.

كتابة الوحدات مشابهة لكتابة أي ملف بايثون آخر. يمكن أن تحتوي الوحدات على تعريفات

الدوال والأصناف والمتغيّرات التي يمكن استخدامها بعد ذلك في برامج بايثون الأخرى.

سننشئ من بيئة البرمجة الحالية الخاصة ببائثون 3 أو بيئة البرمجة المستندة إلى الخادم

ملفًا باسم `hello.py`، والذي سنستورده لاحقًا من ملف آخر.

في البدء، سننشئ دالة تطبع العبارة "Hello, world!":

```
# تعريف دالة
def world():
    print("Hello, world!")
```

إذا نفّذنا البرنامج في سطر الأوامر باستخدام `python hello.py`، فلن يحدث شيء، لأننا

لم نطلب من البرنامج فعل أي شيء.

لننشئ ملفًا ثانيًا في نفس المجلد (أي بجانب الملف السابق) باسم `main_program.py`

حتى نتمكن من استيراد الوحدة التي أنشأناها للتو، ومن ثم استدعاء الدالة. يجب أن يكون هذا

الملف في نفس المجلد حتى تعرف بايثون موضع الوحدة، لأنها ليست وحدة مُضمَّنة ضمن

مكتبة بايثون.

```
# استيراد الوحدة hello
import hello

# استدعاء الدالة
hello.world()
```

نظرًا لأننا استوردنا الوحدة، نحتاج إلى استدعاء الدالة من خلال الإشارة إلى اسم الوحدة بالصياغة النقطية (dot notation). يمكننا بدلاً من ذلك استيراد دالة محدّدة من الوحدة بالتعليمة `from hello import world`، واستدعاء تلك الدالة بالشكل `world()` كما تعلمنا ذلك من الفصل السابق. الآن، يمكننا تنفيذ البرنامج من سطر الأوامر:

```
python main_program.py
```

سنحصل على المخرجات التالية:

```
Hello, world!
```

لنرى كيف يمكننا استخدام المتغيرات في الوحدات، دعنا نضيف تعريفًا لمتغير في

الملف `hello.py`:

```
# تعريف دالة
def world():
    print("Hello, world!")

# تعريف المتغير
shark = "Sammy"
```

بعد ذلك، سنستدعي المتغير داخل الدالة `print()` في الملف `main_program.py`:

```
# استيراد الوحدة
import hello

# استدعاء الدالة
hello.world()

# طباعة المتغير
print(hello.shark)
```

بمجرد تنفيذ البرنامج، سنحصل على المخرجات التالية:

```
Hello, world!
Sammy
```

أخيرًا، دعنا نعرّف صنفًا في الملف `hello.py`. سننشئ الصنف `Octopus`، والذي يحتوي على الخاصيتين `name` و `color`، إضافة إلى دالة تطبع الخاصيات عند استدعائها:

```
# تعريف الدالة
def world():
    print("Hello, world!")

# تعريف المتغير
shark = "Sammy"

# تعريف الصنف
class Octopus:
    def __init__(self, name, color):
        self.color = color
        self.name = name

    def tell_me_about_the_octopus(self):
        print("This octopus is " + self.color + ".")
        print(self.name + " is the octopus's name.")
```

سنضيف الآن الصنف إلى نهاية الملف `main_program.py`:

```
# استيراد الوحدة hello
import hello

# استدعاء الدالة
hello.world()

# طباعة المتغير
print(hello.shark)

# استدعاء الصنف
jesse = hello.Octopus("Jesse", "orange")
jesse.tell_me_about_the_octopus()
```

بمجرد استدعاء الصنف `Octopus` باستخدام `hello.Octopus()`، يمكننا الوصول إلى دوال وخصائص الصنف من فضاء الأسماء الخاص بالملف `main_program.py`. يتيح لنا هذا كتابة `jesse.tell_me_about_the_octopus()` في السطر الأخير دون استدعاء `hello`. يمكننا أيضًا، على سبيل المثال، استدعاء إحدى خصائص الصنف، مثل `jesse.color`، دون الرجوع إلى اسم الوحدة `hello`.

سنحصل عند تنفيذ البرنامج التالي على المخرجات التالية:

```
Hello, world!
Sammy
This octopus is orange.
Jesse is the octopus's name.
```

من المهم أن تضع في الحسبان أن الوحدات لا تضم تعريفات دوال فقط، بل يمكن أيضًا أن

تقدم شيفرات برمجية وتنقذها. لتوضيح هذا، دعنا نعيد كتابة الملف `hello.py` لنجعله يقدم الدالة `world()` وينفذها أيضًا:

```
# تعريف دالة
def world():
    print("Hello, world!")

# استدعاء الدالة داخل الوحدة
world()
```

لقد حذفنا أيضًا التعريفات الأخرى في الملف. الآن، في الملف `main_program.py`، سنحذف كل الأسطر باستثناء عبارة الاستيراد:

```
# استيراد الوحدة hello
import hello
```

عند تنفيذ `main_program.py`، سنحصل على المخرجات التالية:

```
Hello, world!
```

هذا لأن الوحدة `hello` قدمت الدالة `world()`، والتي مُدِّرَتْ بعد ذلك إلى

`main_program.py` لَتُنَفَّذَ مع السكريبت `main_program.py`.

الوحدة هي ملف بايثون مؤلف من تعريفات و شيفرات برمجية يمكن الاستفادة منها في

ملفات بايثون الأخرى.

6. الوصول إلى الوحدات من مجلد آخر

قد تكون الوحدات مفيدة لأكثر من مشروع واحد، وفي هذه الحالة، لن يكون من الحكمة

الاحتفاظ بالوحدة في مجلد مرتبط بمشروع خاص.

إذا أردت استخدام وحدة من مجلد آخر غير المجلد الذي يحوي البرنامج الرئيسي، فأمامك عدّة خيارات سنسردها فيما يلي.

١. التعرف تلقائيًا على مسار الوحدة

أحد الخيارات هو استدعاء مسار الوحدة من الملفات البرمجية التي تستخدم تلك الوحدة. يُعد هذا حلًا مؤقتًا يمكن استخدامه أثناء عملية التطوير، لأنّه لا يجعل الوحدة متاحة على مستوى النظام بأكمله. لإلحاق مسار وحدة بملف برمجي آخر، ستبدأ باستيراد الوحدة `sys`، إلى جانب الوحدات الأخرى التي ترغب في استخدامها في ملف البرنامج الرئيسي.

تعد الوحدة `sys` جزءًا من مكتبة بايثون القياسية، وتوفر معاملات ودوال نظامية يمكنك استخدامها في برنامجك لتعيين مسار الوحدة التي ترغب في تقديمها. على سبيل المثال، لنقل أننا نقلنا الملف `hello.py` إلى المسار `/usr/sammy/`، بينما يوجد الملف `main_program.py` في مجلد آخر. في الملف `main_program.py`، ما يزال بإمكاننا استيراد الوحدة `hello` عن طريق استيراد الوحدة `sys`، ثم إضافة المسار `/usr/sammy/` إلى المسارات التي يبحث بايثون فيها عن الملفات.

```
import sys
sys.path.append('/usr/sammy/')
import hello
...
```

إن عيّنت مسار الملف `hello.py` كما يجب، فسيكون بمقدورك تنفيذ الملف `main_program.py` دون أيّ أخطاء، وستحصل على نفس المخرجات التي حصلنا عليها أعلاه عندما كان الملف `hello.py` في نفس المجلد.

ب. إضافة الوحدة إلى مسار بايثون

الخيار الثاني هو إضافة الوحدة إلى المسار الذي يبحث فيه بايثون عن الوحدات والحزم، وهذا حل أفضل وأدوم، إذ يجعل الوحدة متاحةً على نطاق البيئة، أو على مستوى النظام. لمعرفة المسار الذي يبحث فيه بايثون، شغّل مترجم (interpreter) بايثون من بيئة البرمجة خاصتك:

```
python
```

بعد ذلك، استورد الوحدة `sys`:

```
import sys
```

ثم اطلب من بايثون طباعة مسار النظام:

```
print(sys.path)
```

ستحصل على بعض المخرجات، وسيُطبع مسار نظام واحد على الأقل. إذا كنت تعمل في بيئة برمجة، فقد تتلقى العديد منها. سيكون عليك البحث عن المسارات الموجودة في البيئة التي تستخدمها حالياً، ولكن قد ترغب أيضاً في إضافة الوحدة إلى مسار النظام الرئيسي لبائثون. النتيجة ستكون مشابهة لما يلي:

```
'/usr/sammy/my_env/lib/python3.5/site-packages'
```

يمكنك الآن نقل الملف `hello.py` إلى هذا المجلد. بعد ذلك، يمكنك استيراد الوحدة `hello`

مثل المعتاد:

```
import hello
...
```

عند تنفيذ البرنامج السابق، يُفترض ألا يحدث أي خطأ. يضمن لك تعديل مسار الوحدة إمكانية الوصول إليها مهما كان المجلد الذي تعمل فيه، إذ هذا مفيد خاصة في حال كنت تعمل على عدة مشاريع تشير إلى الوحدة نفسها.

7. خلاصة الفصل

يسمح مفهوم الوحدات باستدعاء دوال غير مضمّنة في بايثون. فبعض الوحدات مُثبّنة كجزء من بايثون، وبعضها سنثبّتها عبر pip، ويتيح لنا استخدام الوحدات توسيع برامجنا وتدعيمها، إذ تضع تحت تصرّفنا شفرات جاهزة للاستخدام، فيمكننا إنشاء وحدات خاصة بنا، لنستخدمها نحن، أو المبرمجون الآخرون.